

# Macro Copilot

An LLM-orchestrated research platform for macro funds, built on a typed substrate of validated quantitative primitives.

---

## 01 The problem

Macro hedge funds run on a specific human workflow. A PM poses a question, an analyst translates it into a sequence of analyses, the PM looks at the output and asks a follow-up. The PM's edge is which questions get asked next; the analyst's value is executing them quickly and correctly. Today this loop is bottlenecked at every stage.

**Bloomberg and Refinitiv** answer single-instrument queries; they cannot compose. *"Show me the 2s10s"* works. *"Show me the 2s10s conditional on QT regimes, regressed against credit spreads, on a rolling 60-day window"* does not.

**Excel and Python notebooks** compose anything, but are write-once and audit-impossible. Reproducing a six-month-old analysis is a project, not a query.

**Internal quant platforms** (Citadel's Aqua, Millennium's research stack) work, but cost hundreds of millions of dollars and decades of engineering, and stay locked inside the firm.

**General-purpose LLMs** can *talk about* the question but cannot execute on your data, cannot guarantee numerical reproducibility, and hallucinate the moment the question gets specific.

The gap is a system that does what an excellent macro analyst does: translate a PM's question into deterministic, lineage-tracked, reproducible quantitative work, and stay numerically honest under arbitrary composition.

## 02 What we are building

Macro Copilot is an LLM-orchestrated research platform with deterministic execution. A PM types a question in plain English. An LLM router translates it into a typed computation graph (DAG) over a closed set of validated primitives. The substrate executes that graph against a multi-asset time-series database. Results stream back into the chat with full lineage, ready for the next question.

The four properties below describe how the system behaves under arbitrary composition.

---

### Composable across asset classes

Any question across rates, FX, credit, commodities, equities and vol (single-instrument, cross-asset, conditional, lagged, regime-switching) is answered by composing primitives already in the registry.

---

### Reproducible by hash

Every output ships with a content-addressed lineage chain back to source rows. Re-running a query six months later produces byte-identical results.

---

### Iterative by construction

*"Now condition on QT regimes"* extends the prior DAG without restarting. Conversational state carries lineage across turns.

---

### Auditable end-to-end

Every primitive's methodology lives in version-controlled YAML. Every workflow's topology is locked, typed, and human-readable.

---

03 How it works

The system is built as five strict layers. Each layer's invariants are protected by the layer below. The LLM never invents new math; it composes the closed registry above.

|    |                    |                                                                                                                                                                                                                  |
|----|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| L1 | Data substrate     | TimescaleDB hypertable, one row per (date, instrument, field). Full instrument master. Audit log of every ingestion event.                                                                                       |
| L2 | Primitives         | Single-purpose, type-safe, deterministic computations (curve spreads, swap spreads, OIS forwards). Each ships its own <i>config.yaml</i> documenting methodology and conventions.                                |
| L3 | Operators          | Finance-blind composition: <i>align_series</i> , <i>series_arithmetic</i> , <i>threshold_events</i> , <i>event_windows</i> , <i>conditional_aggregate</i> . They know about Series and Panels, not about yields. |
| L4 | Workflow substrate | A typed DAG validator and executor. Every workflow is type- and unit-checked before execution. Every node is content-addressed; every result carries its lineage chain.                                          |
| L5 | Workflow templates | Named archetypes (event study, regime-conditioned relationship) that the LLM router binds at runtime from natural-language prompts.                                                                              |

The reasoning stack: what the LLM actually does

Tier 1 · Templates LIVE · RATES

The router matches the prompt to a registered archetype and binds slots. Fast, audit-friendly, regulator-defensible: the math is locked in template YAML.

Tier 2 · Operator DAGs IN DESIGN

For follow-ups that don't match an archetype but compose cleanly, the LLM generates an operator-only graph against a fixed set of primitive outputs. Validated before execution.

Tier 3 · Open generation CONCEPT

The LLM composes any primitive + operator graph against the full registry. Malformed graphs are rejected cheaply by the validator before any compute runs.

**Defensibility lives in the substrate, not the LLM.** A closed primitive registry, finance-blind operators, and a typed DAG validator mean that every LLM-generated graph is checked before it runs, and every result is reproducible by hash. An unconstrained LLM acting on market data offers neither: malformed calls become runtime errors, and numerical answers stay unverifiable.

04 Where we are

|                                                     |            |                                                    |            |
|-----------------------------------------------------|------------|----------------------------------------------------|------------|
| Data substrate (rates: Bloomberg + Refinitiv + ICE) | PRODUCTION | Workflow substrate & lineage                       | PRODUCTION |
| ~10 rates primitives, full methodology discipline   | PRODUCTION | Frontend: workspace, copilot chat, library         | PRODUCTION |
| 7 finance-blind operators, fully typed              | PRODUCTION | Tier 1 templates: event study, regime relationship | PRODUCTION |
| Conversational iteration loop (Tier 2 routing)      | NEXT PHASE | Second domain agent (FX or credit)                 | SCOPING    |

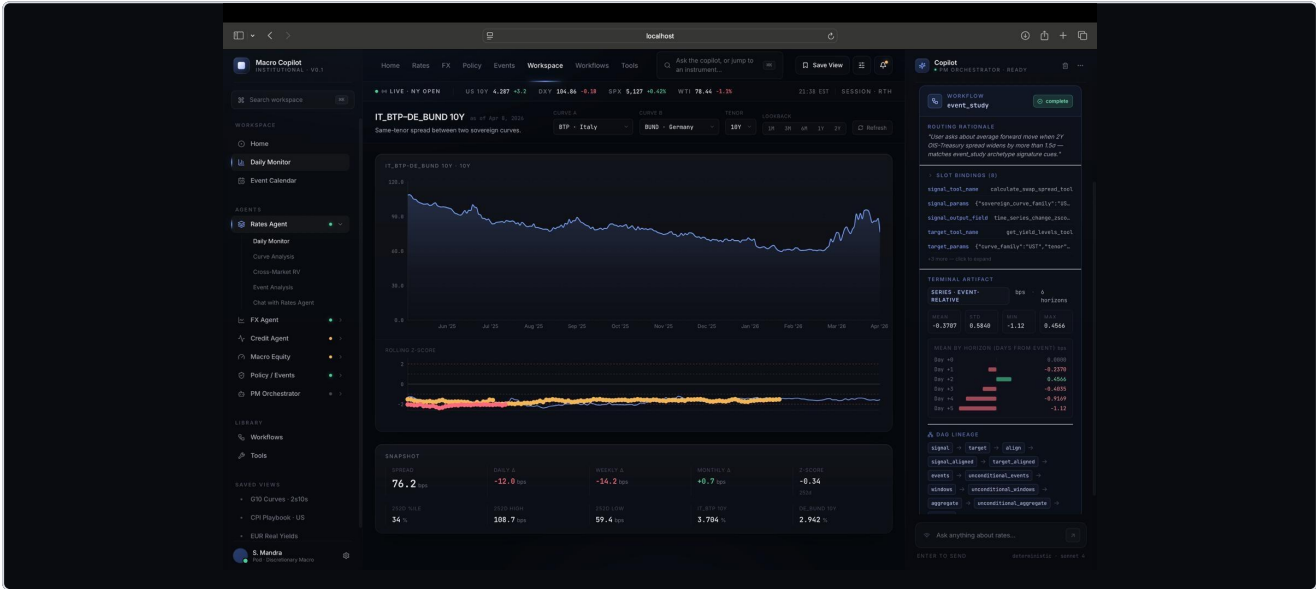
05 Status and next phase

Two milestones are in active development. First, the conversational iteration loop, allowing follow-up turns to extend the prior DAG rather than re-route from scratch. Second, the second domain agent (FX or credit), which validates the substrate's portability beyond rates by reusing the operator catalogue without modification.

# Product preview

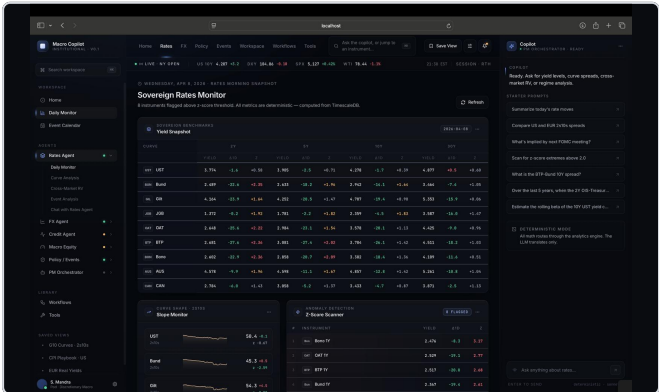
A walk through the surfaces a PM uses day-to-day: chat, workspace, and library.

Three surfaces sit on top of the typed primitive and operator registry: a streamed copilot chat, an interactive workspace for any registered primitive, and a catalogue of every tool and workflow in the system.



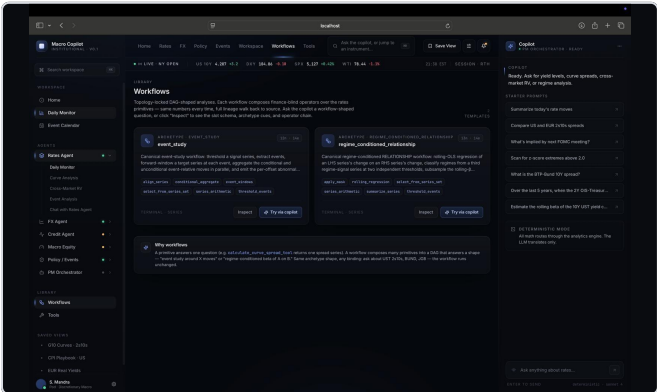
## WORKSPACE AND WORKFLOW RESULT

An *event\_study* workflow run on the BTP versus Bund 10Y spread. The chart sits in the workspace; the right panel shows the executed workflow card with routing rationale, slot bindings, terminal artifact, and the DAG lineage chain that makes the result reproducible by hash.



## DAILY MONITOR

The default rates surface: yield snapshot across G10 sovereigns, curve-shape monitor, and a z-score scanner that flags instruments crossing methodology-defined thresholds.



## WORKFLOWS CATALOGUE

Every registered workflow template is human-readable: the operator chain (align\_series, threshold\_events, event\_windows, conditional\_aggregate) is visible, and "Try via copilot" drops a starter prompt into chat.